

ADR-006: how a fork build updates itself

Status: BUILT AND PUBLISHING, 2026-10-31. Written as a proposal and updated the same day once it worked. Jay asked for "an auto updater for our code" and chose, against the recommendation below, to put it **in the game** rather than beside it. That choice is recorded, and so is what it costs.

Client	`yarg` `a8c35a92` — `Assets/Script/Integration/SelfUpdate/`, a Settings tab, 15 smoke cases
Helper test	`yarg` `97c0bd84` — `tools/test-update-helper.ps1`, 13 checks over 3 paths
Publisher	`yarg` `6744f8f6` — `tools/publish-update.ps1`
Server	vault2 container `yarg-updates` (nginx:alpine), `/mnt/user/YARG/updates`
Manifest	`http://192.168.2.7:8098/dev.json`
First published build	`dev b4183 (6744f8f6)`, 166,133,178 bytes

The port is bound to `192.168.2.7`, not `0.0.0.0`, so the update channel is reachable on the LAN and not over the tailnet or anything SWAG and cloudflared front. That is deliberate and is the mitigation for the unsigned-manifest risk below - it is not a substitute for a signature.

The updater cannot bootstrap itself. A build made before this existed has no updater and will never check for one, so the first install of an updater-capable build on any machine is a manual copy. MSI received `b4183` by SMB on 2026-10-31 for exactly that reason.

What prompted it

The fork is built by hand on ENG-1 and copied to a machine. On 2026-10-31 that machine was `MSI`, and the copy was a 159 MiB zip over SMB. That does not scale past one person who is willing to be told when a build exists.

Measured first, because the obvious plan does not work

A build made by `BuildPlayerHeadless` cannot tell you which build it is.

`GlobalVariables.LoadVersion()` reads `Resources/version.txt` only under `YARG_TEST_BUILD` or `YARG_NIGHTLY_BUILD`. Our headless build sets **no scripting defines** — `BuildOptions.Development` is a build *option*, not a define — so `CurrentVersion` stays the hardcoded `"v0.15"`. An updater that asks the game "what are you?" gets a constant.

Two further consequences of the same fact, both measured on the MSI build:

`PersistentDataPath`	`…/FatalException/YARG-FE/**release**/` — *not* `dev/`, which needs `YARG_TEST_BUILD`
`Assets/Resources/version.txt`	`dev b4180 (679f00d3)` — written on every build by `BuildGitCommitVersion`
that string inside `build/msi/YARG_Data/resources.assets`	**present**

So the identity *ships*; only upstream's accessor refuses to read it. **The updater will read `Resources.Load<TextAsset>("version")` directly** and leave `LoadVersion()` alone. That is both the smaller diff and the one that does not move the data folder out from under an existing install — adding `YARG_TEST_BUILD` to get a readable version would silently relocate settings, song cache and characters from `release/` to `dev/`.

The format is `<branch> b<count> (<shorthash>)`, and `<count>` is `git rev-list --count HEAD`: **monotonic on a branch, and therefore the only field that may be used for ordering.** Comparing hashes tells you *different*, never *newer*.

The recommendation that was not taken

An updater **beside** the game rather than inside it: no Unity code at all, nothing to upstream, and no risk of a half-written client replacing its own files. Jay chose in-game. The reasons against are not fatal, and they are:

1. It is the largest non-upstreamable thing the fork would carry. `RemoteQueue` is defensible to upstream as a feature; "this fork updates itself from Jay's server" is not.
2. **A running process on Windows cannot replace its own files.** In-game therefore still needs an external helper. "In the game" buys the *check and the UI*, not the swap.

So the design below is in-game for everything except the one step that cannot be.

Design

Hosting. A static manifest and zip on vault2, served by the existing web stack — *not* a new endpoint in `yarg-song-server`. The server would need a Go change and a release to gain a feature that nginx already has, and the song server's job is songs.

```
{
  "branch": "dev",
  "build": 4181,
  "commit": "a1b2c3d",
  "url": "http://192.168.2.7:8099/updates/YARG-FE-b4181.zip",
  "sha256": "<64 hex>",
  "size": 166122027,
  "notes": "one line, shown to the player"
}
```

In the game — new code only, in `Assets/Script/Integration/SelfUpdate/`, following `RemoteQueue`'s precedent of owning no scene object and touching nothing existing except one settings row:

1. Read own identity from `Resources/version.txt`. Unparseable → feature disables itself silently.
2. `GET` the manifest on a timeout. **Never block the main menu**; a dead server must cost nothing.
3. Newer iff `manifest.branch == mine && manifest.build > mine`. A *different* branch is never an update.
4. Show it in Settings. Download only on an explicit click — never automatically.
5. Download to `PersistentDataPath/update-staging/`, then **verify sha256 before anything is unpacked**. Mismatch → delete and report. Unpack to a staging folder.
6. Write `update-pending.json` naming the staging folder and the install folder.

The helper — `yarg-update.exe` (or `.cmd`), shipped beside `YARG.exe`:

1. Game spawns it with its own PID and exits.
2. Helper waits for that PID, renames the install folder aside, moves staging into place, relaunches `YARG.exe`, then deletes the old folder. **Any failure restores the rename.**

What this costs, stated plainly

- **An update is arbitrary code execution, over plain HTTP.** A sha256 in the manifest proves the zip matches the manifest; it proves nothing if the manifest itself is spoofed. Anyone who can answer for `192.168.2.7` can run code on every machine running the fork. The honest fixes are HTTPS with a pinned cert, or signing the manifest with a key whose public half is compiled into the client. **Neither is in this design yet, and the feature should not leave the LAN without one.** The song server already accepts plain HTTP on the LAN, but that risk is "someone serves you a bad song", not "someone replaces your executable".
- **603 MB per update**, whole-build replacement. No deltas. Over wifi that is minutes.
- Release binaries are **not signed** (`SYNC-CLIENT.md`), so Windows SmartScreen will have opinions about a freshly swapped exe.
- The fork cannot update a build it did not make: the manifest's `branch` must match, which means a machine on `dev` never silently receives something from another branch.

Decided since

- **The helper is PowerShell**, shipped in `Assets/StreamingAssets/update/` so it lands in a build with no change to the build script, and copied out of the install folder before it runs. A Go binary stays the better long-term answer because it can be signed; this needed no new build step. It is tested against throwaway installs, including the case where the guard that refuses a junk staging folder was removed - which really did destroy the fake install, and is why that test exists.
- **Publishing is a script**, `tools/publish-update.ps1`: build, package, upload, manifest, verify over HTTP. It refuses a dirty tree and takes the build number from the version file the player reads rather than from a separate `git` call.

Still open

- `LoadVersion()` is still upstream's, and still unreadable in our builds. Fixing it is one line and would move every existing install's data folder from `release/` to `dev/`, so it is not worth doing for its own sake.
- **No update has been applied by the game yet.** The helper is tested against fake installs and the manifest is served and verified, but nothing has yet gone check -> download -> restart -> new build on a real machine. Until that happens this is a mechanism, not a feature.
- Signing, which the risk section above argues for and nobody has done.